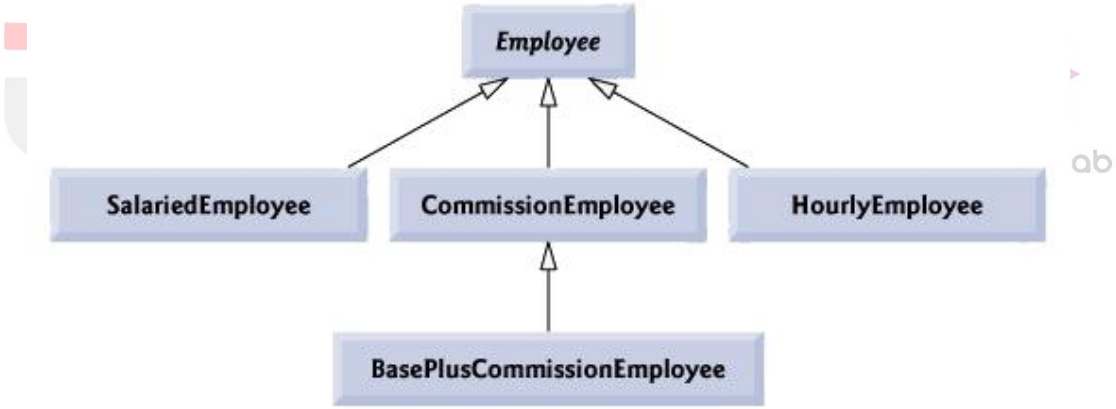
Modul 7 Abstract dan Interface

Tujuan Praktikum
1. Praktikan dapat memahami dan mengerti konsep interface dan mampu mengimplementasikan dalam Java. 2. Praktikan dapat memahami dan mengerti konsep abstract dan mampu mengimplementaskan dalam Java. 3. Praktikan mampu membedakan antara kelas abstract dan *interface*.

7.1 Kelas Abstract

Kata kunci abstrak digunakan untuk *method* atau *class* yang **belum memiliki implementasi**. Abstrak *method* dideklarasikan pada abstrak *class*. Kelas yang dideklarasikan sebagai abstrak tidak akan bisa dibentuk *Object* dalam Java. Jadi kelas abstrak tidak akan bisa digunakan di dalam program Kita yang lain, kecuali kelas abstrak tersebut diturunkan dan turunan dari kelas abstrak tersebut yang bisa dijadikan *Object*. Sebagai ilustrasi dari kelas abstrak perhatikan gambar di bawah ini :



Gambar 7-1 Contoh kelas abstrak dalam UML

Berdasarkan gambar diatas kelas **Employee** digunakan untuk merepresentasikan konsep umum pegawai. Kelas **Employee** diperluas ke kelas **SalariedEmployee, CommissionEmployee dan HourlyEmployee. BasePlusCommissionEmployee** yang memperluas **CommisionEmployee** merepresentasikan tipe akhir pegawai. Berdasarkan gambar kelas **Employee** diketik miring menandakan bahwa kelas tersebut merupakan kelas abstrak.

Berikut contoh program dari kelas abstrak *employee* dan Kelas *SalariedEmployee*.

Kelas Abstrak *Employee*

```
public abstract class Employee
  {
     private String firstName;
     private String lastName;
     private String socialSecurityNumber;
```

```
   // three-argument constructor
   public Employee( String first, String last, String ssn )
   {
      firstName = first;
      lastName = last;
      socialSecurityNumber = ssn;
   } // end three-argument Employee constructor

   // set first name
   public void setFirstName( String first )
   {
      firstName = first;
   } // end Method setFirstName

   // return first name
   public String getFirstName()
   {
      return firstName;
   } // end Method getFirstName

   // set last name
   public void setLastName( String last )
   {
      lastName = last;
   } // end Method setLastName

   // return last name
   public String getLastName()
   {
      return lastName;
   } // end Method getLastName

   // set social security number
   public void setSocialSecurityNumber( String ssn )
   {
      socialSecurityNumber = ssn; // should validate
   } // end Method setSocialSecurityNumber

   // return social security number
   public String getSocialSecurityNumber()
   {
      return socialSecurityNumber;
   } // end Method getSocialSecurityNumber

   // return String representation of Employee Object
   public String toString()
   {
      return String.format( "%s %s\nsocial security number: %s",
         getFirstName(), getLastName(), getSocialSecurityNumber() );
   } // end Method toString

   // abstract Method overridden by subclasses
   public abstract double earnings(); // no implementation here
} // end abstract class Employee
```

Kelas *SalariedEmployee* yang merupakan turunan dari Kelas Abstrak.

```
public class SalariedEmployee extends Employee {

     private double weeklySalary;

     // four-argument constructor
     public SalariedEmployee( String first, String last, String ssn,
        double salary )
     {
        super( first, last, ssn ); // pass to Employee constructor
        setWeeklySalary( salary ); // validate and store salary
     } // end four-argument SalariedEmployee constructor

     // set salary
     public void setWeeklySalary( double salary )
     {
        weeklySalary = salary <0.0 ? 0.0 : salary;
     } // end Method setWeeklySalary

     // return salary
     public double getWeeklySalary()
     {
        return weeklySalary;
     } // end Method getWeeklySalary

     // calculate earnings; override abstract Method earnings in Employee
     public double earnings()
     {
     return getWeeklySalary();
     } // end Method earnings

     // return String representation of SalariedEmployee Object
     public String toString()
     {
     return String.format( "salaried employee: %s\n%s: $%,.2f",
     super.toString(), "weekly salary", getWeeklySalary() );
     } // end Method toString

  } // end class SalariedEmployee
```

7.2 Interface

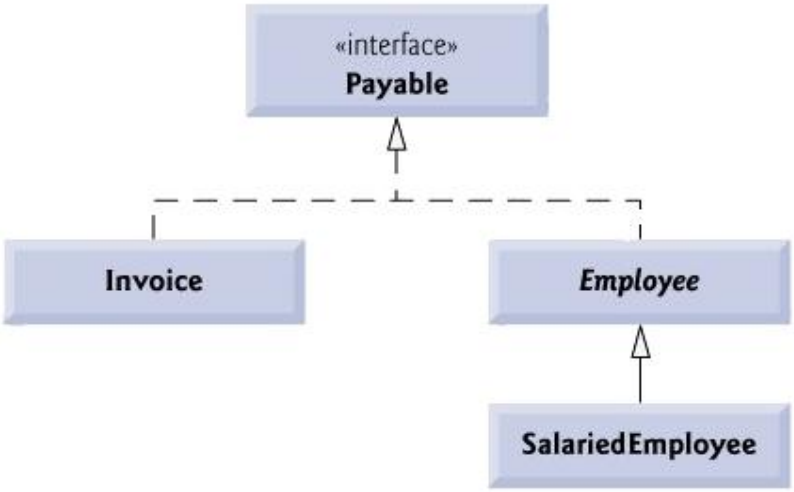
Interface serupa dengan kelas. Kelas mendefinisikan kemampuan sebuah *Object*, sementara *interface* mendefinisikan method atau konstanta yang akan diimplementasikan pada *Object* yang lain. *Interface* membantu mendefinisikan sifat *Object* dengan mendeklarasikan seperangkat karakteristik *Object* tersebut. Sebagai contoh kasus, radio, TV, dan *speaker* komputer memiliki pengontrol volume. Untuk alasan ini, mungkin Kita menginginkan *device-device* ini mengimplementasikan *interface* yang bernama **VolumeControl()**.

Interface pada Java 8 memiliki beberapa batasan:

a. Semua atribut adalah *public, static* dan *final* (semua atribut bertindak sebagai konstanta)
b. Hanya memiliki *default method, dan static method*
c. Tidak boleh ada deklarasi konstruktor.

Berbeda dengan *class* biasa, *interface* mengizinkan **multiple inheritance**, yaitu sebuah kelas dapat mempunyai dua *interface* induk sekaligus. Hal tersebut tidak dapat dilakukan pada sebuah kelas**,** dimana sebuah kelas hanya boleh mempunyai satu kelas induk.

Syntax yang diperlukan hampir sama dengan cara **membuat kelas**. Perbedaannya hanya *method* pada **interface tidak memiliki isi/ variabel.** Sebuah contoh dengan tiga item: *interface*, kelas yang mengimplementasikan *interface*, dan kelas yang menggunakan kelas yang disalurkan. Sebagai ilustrasi perhatikan gambar UML berikut:



Gambar 7-2 Contoh kelas interface dalam UML

Fakultas Informatika
School of Computing
Telkom University

Berdasarkan gambar diatas hierarki dimulai dengan *interface* **Payable**. Relasi UML yang terlihat antara kelas dan interface dinamakan **realization**. Sebuah kelas dikatakan merealisasikan atau mengimplementasikan method pada sebuah **interface**.

Berikut contoh program dari UML diatas (Hanya *interface* Payable dan Kelas Invoice) .

Interface Payable

```
public interface Payable
{
   double getPaymentAmount(); // calculate payment; no implementation
} // end interface Payable
```

Kelas Invoice yang mengimplementasikan Payable

```
public class Invoice implements Payable
  {
     private String partNumber;
     private String partDescription;
     private int quantity;
     private double pricePerItem;

     // four-argument constructor
     public Invoice( String part, String description, int count,
        double price )
     {
        partNumber = part;
        partDescription = description;
```

```
         setQuantity( count ); // validate and store quantity
         setPricePerItem( price ); // validate and store price per item
      } // end four-argument Invoice constructor

      // set part number
      public void setPartNumber( String part )
      {
         partNumber = part;
      } // end Method setPartNumber

      // get part number
      public String getPartNumber()
      {
         return partNumber;
      } // end Method getPartNumber

      // set description
      public void setPartDescription( String description )
      {
       partDescription = description;
      } // end Method setPartDescription
      // get description
      public String getPartDescription()
      {
         return partDescription;
      } // end Method getPartDescription

      // set quantity
      public void setQuantity( int count )
      {
         quantity = ( count <0 ) ? 0 : count; // quantity cannot be
negative
      } // end Method setQuantity

      // get quantity
      public int getQuantity()
      {
         return quantity;
      } // end Method getQuantity

      // set price per item
      public void setPricePerItem( double price )
      {
         pricePerItem = ( price <0.0 ) ? 0.0 : price; // validate price
      } // end Method setPricePerItem

      // get price per item
      public double getPricePerItem()
      {
         return pricePerItem;
      } // end Method getPricePerItem


      // return String representation of Invoice Object
      public String toString()
      {
         return String.format( "%s: \n%s: %s (%s) \n%s: %d \n%s:
$%,.2f",
```

```
           "invoice", "part number", getPartNumber(),
getPartDescription(),
           "quantity", getQuantity(), "price per item",
getPricePerItem() ;
     } // end Method toString

     // Method required to carry out contract with interface Payable
     public double getPaymentAmount()
     {
     return getQuantity() * getPricePerItem(); // calculate total cost
     } // end Method getPaymentAmount
  } // end class Invoice
```

```
public interface Product {
   static final String MAKER = "My Corp";
   static final String PHONE = "555-123-4567";
   public int getPrice(int id);
}

public class Shoe implements Product {
   public int getPrice(int id) {
      if (id == 1)
            return(5);
      else
            return(10);
   }

   public String getMaker() {
      return(MAKER);
   }
}

public class Store {
   static Shoe hightop;
   public static void init() {
      hightop = new Shoe();
   }
}

public static void main(String argv[]) {
   init();
   getInfo(hightop);
   orderInfo(hightop);

   public static void getInfo(Shoe item) {
         System.out.println("This Product is made by "+ item.MAKER);
         System.out.println("It costs $" + item.getPrice(1) + '\n');
   }

   public static void orderInfo(Product item) {
      System.out.println("To    order    from    "+item.MAKER+"    call
"+item.PHONE+".");
```

```
      System.out.println("Each item costs $"+item.getPrice(1));
   }
}
```

Interface sebelum Java 8 hanya dapat memiliki *method* abstrak. Semua *method* di *Interface* bersifat publik dan abstrak secara *default*. Namun pada Java 8 memungkinkan *interface* untuk memiliki *method default* dan *static*. Hal tersebut dikarenakan *default method* pada *interface* dapat memungkinkan pengembang menambahkan *method* baru ke *interface* tanpa mempengaruhi kelas yang mengimplementasikan *interface*.

Default method pada *Interface* :

```
interface MyInterface{
    default void newMethod(){
        System.out.println("Newly added default method");
    }
    void existingMethod(String str);
}
public class Example implements MyInterface{
    // implementasi abstract method
    public void existingMethod(String str){
        System.out.println("String is: "+str);
    }

    public static void main(String[] args) {
      Example obj = new Example();
      obj.newMethod();
       obj.existingMethod("Java 8 is easy to learn");
    }
}
```

Method **newMethod** pada *interface* **MyInterface** yang berarti kita tidak perlu mengimplementasikan *method* tersebut di kelas implementasi **Example**. Dengan menggunakan metode ini kita dapat menambahkan *method default* ke dalam *interface* tanpa perlu khawatir dengan kelas yang mengimplementasikan *interface* tersebut.

Static method pada *Interface* :

```
interface MyInterface{
    default void newMethod(){
        System.out.println("Newly added default method");
    }
    static void anotherNewMethod(){
      System.out.println("Newly added static method");
    }
    void existingMethod(String str);
}
```

```
public class Example implements MyInterface{
    // implementing abstract method
    public void existingMethod(String str){
        System.out.println("String is: "+str);
    }

    public static void main(String[] args) {
      Example obj = new Example();

       obj.newMethod();
       MyInterface.anotherNewMethod();
       obj.existingMethod("Java 8 is easy to learn");
    }
}
```

Static method pada *interface* hampir sama dengan *default method* sehingga kita tidak perlu mengimplementasikannya di kelas **Example**. Kita dapat dengan aman menambahkannya ke *interface* yang ada tanpa mengubah kode di kelas **Example**. Karena *static method* tidak dapat menimpanya di kelas **Example**.

Perbedaan **abstract class** dan **interface** antara lain:

a. Semua interface Method tidak memiliki body sedangkan beberapa abstract class dapat memiliki Method dengan implementasi.
b. Sebuah *interface* hanya dapat mendefinisikan *constant* sedangkan sebuah *abstract class* tampak seperti kelas biasa yang dapat mendeklarasikan variabel.
c. *Interface* tidak memiliki hubungan *inheritance* secara langsung dengan sebuah *kelas* tertentu, sedangkan *abstract class* bisa jadi hasil turunan dari *abstract class* induknya.
d. *Interface* memungkinkan terjadinya pewarisan jamak (*multiple inheritance*) sedangkan *abstract class* tidak.